大数据应用人才培养系列教材

Python语言

刘 鹏　张 燕　○总主编
李肖俊 ○主编　　刘 河　钟 涛 ○副主编

清華大學出版社
TSINGHUA UNIVERSITY PRESS

第四章 组合数据类型

4.1 列表

4.2 元组

4.3 字典

4.4 集合

4.5 实验

4.6 小结

习题

4.1.1 创建列表

列表（Lists）属于Python中的序列类型，它是任意对象的有序集合，通过“位置”或者“索引”访问其中的元素，它具有可变对象、可变长度、异构和任意嵌套的特点。

4.1.1 创建列表

列表里第一个元素的“位置”或者“索引”是从“0”开始，第二个元素的则是“1”，以此类推。

在创建列表时，列表元素放置在方括号[] 中，以逗号来分隔各元素，格式如下：

```
listname = [元素1, 元素2, 元素3, ……, 元素n]
```

举例如下：

```
sample_list1 = [0, 1, 2, 3, 4]
sample_list2 = ["P", "y", "t", "h", "o", "n"]
sample_list3 = ['Python', 'sample', 'list', 'for', 'your', 'reference']
```

4.1.1 创建列表

代码运行如下：

```
>>> sample_list1 = [0, 1, 2, 3, 4]          #列表sample_list1
>>> sample_list2 = ["P", "y", "t", "h", "o", "n"]  #列表sample_list2
>>> sample_list3 = ['Python', 'sample', 'list', 'for', 'your', 'reference']
#列表sample_list3
>>> print (sample_list1)                    #打印输出列表
[0, 1, 2, 3, 4]                             #输出结果
>>> print (sample_list2)                    #打印输出列表
['p', 'y', 't', 'h', 'o', 'n']              #输出结果
>>> print (sample_list3)                    #打印输出列表
['Python', 'sample', 'list', 'for', 'your', 'reference']  #输出结果
```

4.1.1 创建列表

列表中允许有不同数据类型的元素，例如：

```
sample_list4 = [0, "y", 2, "h", 4, "n", 'Python']
```

但通常建议列表中元素最好使用相同的数据类型。

列表可以嵌套使用，例如：

```
sample_list5 = [sample_list1, sample_list2, sample_list3]
```

运行结果如下：

```
>>> sample_list1 = [0, 1, 2, 3, 4]
>>> sample_list2 = ["P", "y", "t", "h", "o", "n"]
>>> sample_list3 = ['Python', 'sample', 'list', 'for', 'your', 'reference']
>>> sample_list5 = [sample_list1, sample_list2, sample_list3] #创建一个嵌套列表
>>> print (sample_list5)
[[0, 1, 2, 3, 4], ['P', 'y', 't', 'h', 'o', 'n'], ['Python', 'sample', 'list', 'for', 'your', 'reference']]
```

4.1.2 使用列表

通过使用“位置”或者“索引”来访问列表中的值，将放在方括号中。特别注意，“位置”或者“索引”是从0开始，例如引用上一节列表示例sample_list1中的第1个，可以写成：sample_list1[0]；引用第3个值，可以写成：sample_list1[2]。代码示例为：

```
>>> sample_list1 = [0, 1, 2, 3, 4]
>>> print ("sample_list1[0]: ", sample_list1[0])  #输出索引为0的元素
sample_list1[0]:  0
>>> print ("sample_list1[2]: ", sample_list1[2])  #输出索引为2的元素
sample_list1[2]:  2
```

4.1.2 使用列表

可以在方括号中使用“负整数”，如：sample_list1[-2]，意为从列表的右侧开始倒数2个的元素，即索引倒数第2的元素。

```
>>> sample_list1 = [0, 1, 2, 3, 4]
>>> print ("sample_list1[-2]: ", sample_list1[-2])#输出索引倒数第2的元素
sample_list1[-2]:  3
```

4.1.2 使用列表

以在方括号中用冒号分开的两个整数来截取列表中的元素，例如sample_list2[2:4]，可取得列表sample_list2中的第3个和第4个元素，不包含第5个元素。

```
>>> sample_list2 = ["p", "y", "t", "h", "o", "n"]
>>> print ("sample_list2[2:4]:", sample_list2[2:4])
sample_list2[2:4]: ['t', 'h']
```

该类操作被称为“切片”操作（slice）。

4.1.2 使用列表

对列表的元素进行修改时，可以使用赋值语句：

```
>>> sample_list3 = ['python', 'sample', 'list', 'for', 'your', 'reference']
>>> sample_list3[4] = 'my'
>>> print ("sample_list3[4]:", sample_list3[4])
sample_list3[4]: my
>>> print ("sample_list3:", sample_list3)
sample_list3: ['python', 'sample', 'list', 'for', 'my', 'reference']
```

4.1.3 删除列表元素

删除列表的元素，可以使用 del 语句，格式为：

del listname[索引]

该索引的元素被删除后，后面的元素将会自动移动并填补该位置。

在不知道或不关心元素的索引时，可以使用列表内置方法remove()来删除指定的值，例如：

listname.remove('值')

清空列表，可以采用重新创建一个与原列表名相同的空列表的方法，例如：

listname = []

删除整个列表，也可以使用del语句，格式为：

del listname

4.1.3 删除列表元素

代码示例如下：

```
>>> sample_list4 = [0, "y", 2, "h", 4, "n", 'Python']
>>> del sample_list4[5]                    #删除列表中索引为5的元素
>>> print ("after deletion, sample_list4: ", sample_list4)
after deletion, sample_list4:  [0, 'y', 2, 'h', 4, 'Python']
>>> sample_list4.remove('Python')       #删除列表中值为Python的元素
>>> print ("after removing, sample_list4: ", sample_list4)
after removing, sample_list4:  [0, 'y', 2, 'h', 4]
>>> sample_list4 = []                    #重新创建列表并置为空
>>> print (sample_list4)                  #输出该列表
[]
>>> del sample_list4                     #删除整个列表
>>> print (sample_list4)                  #打印输出整个列表
Traceback (most recent call last):
  File "<pyshell#108>", line 1, in <module>
    print (sample_list4)
NameError: name 'sample_list4' is not defined #系统报告该列表未定义
```

4.1.4 列表的内置函数与其他方法

函数	说明
len(listname)	返回列表的元素数量
max(listname)	返回列表中元素的最大值
min(listname)	返回列表中元素的最小值
list(tuple)	将元组转换为列表

代码示例如下：

```
>>> sample_list1 = [0, 1, 2, 3, 4]
>>> len(sample_list1)        #列表的元素数量
5
>>> max(sample_list1)        #列表中元素的最大值
4
>>> min(sample_list1)        #列表中元素的最小值
0
```

注意：Python 3 中已经没有了Python 2中用于列表比较的cmp函数。

4.1.4 列表的内置函数与其他方法

方法	说明
listname.append(元素)	添加新的元素在列表末尾
listname.count(元素)	统计该元素在列表中出现的次数
listname.extend(序列)	追加另一个序列类型中的多个值，到该列表末尾（用新列表扩展原来的列表）
listname.index(元素)	从列表中找出某个值第一个匹配元素的索引位置
listname.insert(位置, 元素)	将元素插入列表
listname.pop([index=-1])	移除列表中的一个元素（“-1”表示从右侧数第一个元素，也就是最后一个索引的元素），并且返回该元素的值
listname.remove(元素)	移除列表中的第一个匹配某个值的元素
listname.reverse()	将列表中元素反向
listname.sort(cmp=None, key=None, reverse=False)	对列表进行排序
listname.clear()	清空列表
listname.copy()	复制列表

第四章 组合数据类型

4.1 列表

4.2 元组

4.3 字典

4.4 集合

4.5 实验

4.6 小结

习题

4.2.1 创建元组

元组（Tuples）与列表一样，属于Python中的序列类型，它是任意对象的有序集合，通过“位置”或者“索引”访问其中的元素，它具有可变长度、异构和任意嵌套的特点，与列表不同的是：元组中的元素是不可修改的。

4.2.1 创建元组

元组的创建很简单，把元素放入小括号，并在每两个元素中间使用逗号隔开即可，格式为：

tuplename = (元素1, 元素2, 元素3, ……, 元素n)

举例如下：

```
sample_tuple1 = (1, 2, 3, 4, 5, 6)
sample_tuple2 = "p", "y", "t", "h", "o", "n"
sample_tuple3 = ('python', 'sample', 'tuple', 'for', 'your', 'reference')
sample_tuple4 = ('python', 'sample', 'tuple', 1989, 1991, 2018)
```

4.2.1 创建元组

元组也可以为空：

```
sample_tuple5 = ()
```

需要注意的是，为避免歧义，当元组中只有一个元素时，必须在该元素后加上逗号，否则括号会被当作运算符，例如：

```
sample_tuple6 = (123,)
```

元组中的元素可以是各种可迭代的数据类型。

4.2.1 创建元组

元组也可以嵌套使用，例如：

```
>>> sample_tuple1 = (1, 2, 3, 4, 5, 6)
>>> sample_tuple2 = "P", "y", "t", "h", "o", "n"
>>> sample_tuple7 = (sample_tuple1, sample_tuple2)
>>> print (sample_tuple7)
((1, 2, 3, 4, 5, 6), ('P', 'y', 't', 'h', 'o', 'n'))
```

4.2.2 使用元组

与列表相同，我们可以通过使用“位置”或者“索引”来访问元组中的值，“位置”或者“索引”也是是从0开始，例如：

sample_tuple1 = (1, 2, 3, 4, 5, 6)

sample_tuple1[1]表示元组tuple1中的第2个元素:2。

sample_tuple1[3:5] 表示元组sample_tuple1中的第4个和第5个元素，不包含第6个元素：4,5。

sample_tuple1[-2] 表示元组sample_tuple1中从右侧向左数的第2个元素：5。

4.2.2 使用元组

代码示例为：

```
>>> sample_tuple1 = (1, 2, 3, 4, 5, 6)
>>> print (sample_tuple1[1])    #截取第2个元素
2
>>> print (sample_tuple1[3:5])   #第4个和第5个元素，不包含第6个元素
(4, 5)
>>> print (sample_tuple1[-2])    #从右侧向左数的第2个元素
5
```

4.2.2 使用元组

元组也支持“切片”操作，例如

sample_tuple2 = "P", "y", "t", "h", "o", "n"

sample_tuple2[:] 表示取元组sample_tuple2的所有元素；

sample_tuple2[3:] 表示取元组sample_tuple2的索引为3的元素之后的所有元素；

sample_tuple2[0:4:2] 表示元组sample_tuple2的索引为0到4的元素，每隔一个元素取一个。

4.2.2 使用元组

代码示例为：

```
>>> sample_tuple2 = "P", "y", "t", "h", "o", "n"
>>> print (sample_tuple2[:])        #取元组sample_tuple2的所有元素
('P', 'y', 't', 'h', 'o', 'n')
>>> print (sample_tuple2[3:])       #取元组的第4个元素之后的所有元素
('h', 'o', 'n')
>>> print (sample_tuple2[0:4:2])    #元组sample_tuple2的第1个到第5元素，每隔一个元素取一个
('P', 't')
```

4.2.3 删除元组

由于元组中的元素是不可变的，也就是不允许被删除的，但可以使用del 语句删除整个元组：

del tuple

代码示例如下：

```
>>> sample_tuple3 = ('Python', 'sample', 'tuple', 'for', 'your', 'reference')
>>> print (sample_tuple3)          #输出删除前的元组sample_tuple3
('Python', 'sample', 'tuple', 'for', 'your', 'reference')
>>> del sample_tuple3              #删除元组sample_tuple3
>>> print (sample_tuple3)          #输出删除后的元组sample_tuple3
Traceback (most recent call last):
  File "<pyshell#49>", line 1, in <module>
    print (sample_tuple3)
NameError: name ' sample_tuple3' is not defined #系统正常报告sample_tuple3没有定义
```

4.2.4 元组的内置函数

函数	说明
len(tuplename)	返回元组的元素数量
max(tuplename)	返回元组中元素的最大值
min(tuplename)	返回元组中元素的最小值
tuple(listname)	将列表转换为元组

4.2.4 元组的内置函数

代码示例如下：

```
>>> sample_tuple1 = (1, 2, 3, 4, 5, 6)    #创建元组tuple1
>>> print (len(sample_tuple1))            #输出元组长度
6
>>> print (max(sample_tuple1))            #输出元组最大值
6
>>> print (min(sample_tuple1))            #输出元组最小值
1
>>> a = [1,2,3]                           #创建列表a
>>> print (a)                             #输出列表a
[1, 2, 3]
>>> print (tuple(a))                      #转换列表a为元组后输出
(1, 2, 3)
```

第四章 组合数据类型

4.1 列表

4.2 元组

4.3 字典

4.4 集合

4.5 实验

4.6 小结

习题

4.3.1 创建字典

字典（Dictionaries），属于映射类型，它是通过键实现元素存取，具有无序、可变长度、异构、嵌套和可变类型容器等特点。

4.3.1 创建字典

字典中的键和值有单引号，他们成对出现，中间用冒号分割，每对直接用逗号分割，并放置在花括号中，格式如下：

dictname = {键1: 值1, 键2: 值2, 键3: 值3, ……, 键n: 值n}

在同一个字典中，键应该是唯一的，但值则无此限制。

举例如下：

```
sample_dict1 = {'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ'}
sample_dict2 = {12: 34, 34: 56, 56: 78}
sample_dict3 = {'Hello': 'World', 34: 56, 'City': 'CQ'}
```

4.3.1 创建字典

创建字典时，同一个键被两次赋值，那么第一个值无效，第二个值被认为是该键的值。

```
sample_dict4 = {'Model': 'PC', 'Brand': 'Lenovo', 'Brand': 'Thinkpad'}
```

这里的键Brand生效的值是Thinkpad。

4.3.1 创建字典

创建字典时，同一个键被两次赋值，那么第一个值无效，第二个值被认为是该键的值。

```
sample_dict4 = {'Model': 'PC', 'Brand': 'Lenovo', 'Brand': 'Thinkpad'}
```

这里的键Brand生效的值是Thinkpad。

4.3.1 创建字典

字典也支持嵌套，格式如下：

```
dictname = {键1: {键11: 值11, 键12: 值12 },
键2:{ 键21: 值21, 键2: 值22},
……,
键n: {键n1: 值n1, 键n2: 值n2}}
```

例如：

```
sample_dict5 = {'office':{ 'room1':'Finance ', 'room2':'logistics'},
                'lab':{'lab1':'Physics', 'lab2':'Chemistry'}}
```

4.3.2 使用字典

使用字典中的值时，只需要把对应的键放入方括号，格式为：
dictname[键]
举例如下：

```
>>> sample_dict1 = {'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ'}
>>> print ("sample_dict1['Hello']: ", sample_dict1['Hello'])
sample_dict1['Hello']:  World                    #输出键为Hello的值
>>> sample_dict2 = {12: 34, 34: 56, 56: 78}
>>> print ("sample_dict2[12]: ", sample_dict2[12])
sample_dict2[12]:  34                            #输出键为12的值
```

4.3.2 使用字典

使用包含嵌套的字典，例如：

```
>>> sample_dict5 = {'office':{ 'room1':'Finance', 'room2':'logistics'},
                    'lab':{'lab1':' Physics', 'lab2':'Chemistry'}}
>>> print ("sample_dict5['office']: ", sample_dict5['office'])
sample_dict5['office']:  {'room1': 'Finance', 'room2': 'logistics'}  #输出键为office 的值
```

4.3.2 使用字典

可以对字典中的已有的值进行修改，例如：

```
>>> sample_dict1 = {'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ'}
>>> print (sample_dict1['City'])            #输出键为City的值
CQ
>>> sample_dict1['City'] = 'NJ'             #把键为City的值修改为NJ
>>> print (sample_dict1['City'])            #输出键为City的值
NJ
>>> print (sample_dict1)
{'Hello': 'World', 'Capital': 'BJ', 'City': 'NJ'}    #输出修改后的字典
```

4.3.2 使用字典

可以向字典末尾追加新的键值，例如：

```
>>> sample_dict1 = {'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ'}
>>> sample_dict1['viewspot']= 'HongYaDong' #把新的键和值添加到字典
>>> print (sample_dict1)                    #输出修改后的字典
{'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ', 'viewspot': 'HongYaDong'}
```

4.3.3 删除元素和字典

可以使用del语句删除字典中的键和对应的值，格式为：

del dictname[键]

使用del语句删除字典，格式为：

del dictname

举例如下：

```
>>> sample_dict1 = {'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ'}
>>> del sample_dict1['City']     #删除字典中的键City和对应的值
>>> print (sample_dict1)        #打印结果
{'Hello': 'World', 'Capital': 'BJ'}
>>> del sample_dict1            #删除该字典
>>> print (sample_dict1)          #打印该字典
Traceback (most recent call last):  #系统正常报错，该字典未定义
  File "<pyshell#71>", line 1, in <module>
    print (sample_dict1)
NameError: name 'sample_dict1' is not defined
```

4.3.4 字典的内置函数和方法

函数	说明
len(distname)	计算键的总数
str(distname)	输出字典
type(distname)	返回字典类型

举例如下：

```
>>> sample_dict1 = {'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ'}
>>> len(sample_dict1)          #计算该字典中键的总数
3
>>> str(sample_dict1)          #输出字典
"{'Hello': 'World', 'Capital': 'BJ', 'City': 'CQ'}"
>>> type(sample_dict1)         #返回数据类型
<class 'dict'>
```

4.3.4 字典的内置函数和方法

方法	说明
dictname.clear()	删除字典所有元素，清空字典
dictname.copy()	以字典类型返回某个字典的浅复制
dictname.fromkeys(seq[, value])	创建一个新字典，以序列中的元素做字典的键，值为字典所有键对应的初始值
dictname.get(value, default=None)	返回指定键的值，如果值不在字典中返回default值
key in dictname	如果键在字典dict里返回true，否则返回false
dictname.items()	以列表返回可遍历的(键, 值) 元组数组
dictname.keys()	将一个字典所有的键生成列表并返回
dictname.setdefault(value, default=None)	和dictname.get()类似，不同点是，如果键不存在于字典中，将会添加键并将值设为default对应的值
dictname.update(dictname2)	把字典dictname2的键/值对更新到dictname里
dictname.values()	以列表返回字典中的所有值
dictname.pop(key[,default])	弹出字典给定键所对应的值，返回值为被删除的值。键值必须给出。 否则，返回default值。
dictname.popitem()	弹出字典中的一对键和值(一般删除末尾对)，并删除

第四章 组合数据类型

4.1 列表

4.2 元组

4.3 字典

4.4 集合

4.5 实验

4.6 小结

习题

4.4.1 创建集合

集合（set），是一种集合类型，可以理解为就是数学课里学习的集合。它是一个可以表示任意元素的集合，它的索引可以通过另一个任意键值的集合进行，它可以无序排列、哈希。

集合分为两类：可变集合（set），不可变集合（frozenset）。

可变集合，在被创建后，可以通过很多种方法被改变，例如add()，update()等。

不可变集合，由于其不可变特性，它是可哈希的（hashable，意为一个对象在其生命周期中，其哈希值不会改变，并可以和其他对象做比较），也可以作为一个元素被其他集合使用，或者作为字典的键。

4.4.1 创建集合

使用大括号 { } 或者set()创建非空集合，格式为：

```
sample_set = {值1, 值2, 值3, ……, 值n}
```

或

```
sample_set = set([值1, 值2, 值3, ……, 值n])
```

创建一个不可变集合，格式为：

```
sample_set = frozenset([值1, 值2, 值3, ……, 值n])
```

举例如下：

```
sample_set1 = {1, 2, 3, 4, 5}
sample_set2 = {'a', 'b', 'c', 'd', 'e'}
sample_set3 = {'Beijing', 'Tianjin', 'Shanghai', 'Nanjing', 'Chongqing'}
sample_set4 = set([11, 22, 33, 44, 55])
sample_set5 = frozenset(['CHS', 'ENG', '', '', '',])    #创建不可变集合
```

但创建空集合时必须使用set()，格式：

```
emptyset = set()
```

4.4.2 使用集合

集合的一个显著的特点就是可以去掉重复的元素，例如：

```
>>> sample_set6 = {1, 2, 3, 4, 5, 1, 2, 3, 4,}
>>> print (sample_set6)                #输出去掉重复的元素的集合
{1, 2, 3, 4, 5}
```

可以使用len()函数来获得集合中元素的数量，例如：

```
>>> sample_set6 = {1, 2, 3, 4, 5, 1, 2, 3, 4,}
>>> len(sample_set6)                #输出集合的元素数量
5
```

注意：这里集合的元素数量，是去掉重复元素之后的数量。

4.4.2 使用集合

集合是无序的，因此没有“索引”或者“键”来指定调用某个元素，但可以使用for循环输出集合的元素，例如：

```
>>> sample_set6 = {1, 2, 3, 4, 5, 1, 2, 3, 4,}
>>> for x in sample_set6:
print (x)
1
2
3
4
5
```

注意：这里输出的集合的元素，也是去掉重复元素之后的。

4.4.2 使用集合

向集合中添加一个元素，可以使用add()方法，即把需要添加的内容作为一个元素（整体），加入到集合中，格式为：

setname.add(元素)

向集合中添加多个元素，可以使用update()方法，将另一个类型中的元素拆分后，添加到原集合中，格式为：

setname.update(others)

两种增加集合元素的方法，对可变集合有效，例如：

```
>>> sample_set1 = {1, 2, 3, 4, 5}
>>> sample_set1.add(6)            #使用add方法添加元素到集合
>>> print ("after being added, the set is: ", sample_set1)
after being added, the set is:  {1, 2, 3, 4, 5, 6}
>>> sample_set1.update('python')     #使用update方法添加另一个集合
>>> print ("after being updated, the set is:", sample_set1)
after being updated, the set is: {'y', 1, 2, 3, 4, 5, 6, 'p', 't', 'n', 'o', 'h', }
```

4.4.2 使用集合

集合可以被用来做成员测试，使用in或not in检查某个元素是否属于某个集合，例如：

```
>>> sample_set1 = {1, 2, 3, 4, 5}
>>> sample_set2 = {'a', 'b', 'c', 'd', 'e'}
>>> 3 in sample_set1            #判断3是否在集合中，是则返回True
True
>>> 'c' not in sample_set2      #判断“c没有在集合中”
False                           #如果c在该集合中，返回False，
#否则返回True
```

4.4.2 使用集合

集合之间可以做集合运算，求差集（difference）、并集（union）、交集（intersection）、对称差集（symmetric difference）

```
>>> sample_set7 = {'C', 'D', 'E', 'F', 'G'}
>>> sample_set8 = {'E', 'F', 'G', 'A', 'B'}
>>> sample_set7 - sample_set8                    #差集
{'D', 'C'}
>>> sample_set7 | sample_set8                    #并集
{'A', 'G', 'B', 'F', 'E', 'D', 'C'}
>>> sample_set7 & sample_set8                    #交集
{'E', 'G', 'F'}
>>> sample_set7 ^ sample_set8                    #对称差集
{'A', 'B', 'D', 'C'}
```

4.4.3 删除元素和集合

可以使用remove()方法删除集合中的元素，格式为：

setname.remove(元素)

可使用del方法删除集合，格式为：

del setname

举例如下：

```
>>> sample_set1 = {1, 2, 3, 4, 5}
>>> sample_set1.remove(1)          #使用remove方法删除元素
>>> print (sample_set1)
{2, 3, 4, 5}
>>> sample_set1.clear()
>>> print (sample_set1)            #清空集合中的元素
set()                              #返回结果为空集合
>>> del sample_set1                #删除集合
>>> print (sample_set1)
Traceback (most recent call last):     #系统报告，该集合未定义
  File "<pyshell#64>", line 1, in <module>
    print (sample_set1)
NameError: name 'sample_set1' is not defined
```

4.4.4 集合的方法

方法	说明
len(ss)	返回集合的元素个数
x in ss	测试x是否是集合ss中的元素，返回True或False
x not in ss	如果x不在集合ss中，返回True，否则返回False
ss.isdisjoint(otherset)	当集合ss与另一集合otherset不相交时，返回True，否则返回False
ss.issubset(otherset) 或 ss <= otherset	如果集合ss是另一集合otherset的子集，返回True，否则返回False
ss < otherset	如果集合ss是另一集合otherset的真子集，返回True，否则返回False
ss.issuperset(otherset) 或 ss >= otherset	如果集合ss是另一集合otherset的父集，返回True，否则返回False
ss > otherset	如果集合ss是另一集合otherset的父集，且otherset是ss的子集，则返回True，否则返回False
ss.union(*othersets) 或 ss | otherset1 | otherset2 …	返回ss和othersets的并集，包含有set和othersets的所有元素
ss.intersection(*othersets)或 ss & otherset1 & otherset2 …	返回ss和othersets的交集，包含在ss并且也在othersets中的元素
ss.difference(*othersets) 或 ss - otherset1 - otherset2 …	返回ss与othersets的差集，只包含在ss中但不在othersets中的元素
ss.symmetric_difference(otherset) 或 set ^ otherset	返回ss与otherset的对称差集，只包含在ss中但不在othersets中，和不在ss中但在othersets中的元素
ss.copy()	返回集合ss的浅拷贝

4.4.4 集合的方法

方法	说明
ss.update(*othersets) 或 ss |= otherset1 | otherset2 …	将另外的一个集合或多个集合元素，添加到集合ss中
ss.intersection_update(*othersets) 或 set &= otherset1 & otherset2 …	在ss中保留它与其他集合的交集
ss.difference_update(*othersets) 或 ss -= otherset1 | otherset2 …	从ss中移除它与其他集合的交集，保留不在其他集合中的元素
ss.symmetric_difference_update(otherset) 或 ss ^= otherset	集合ss与另一集合otherset交集的补集，将结果返回到ss
ss.add(元素)	向集合ss中添加元素
ss.remove(元素)	从集合ss中移除元素，如果该元素不在ss中，则报告KeyError
ss.discard(元素)	从集合ss中移除元素，如果该元素不在ss中，则什么都不做
ss.pop()	移除并返回集合ss中的任一元素，如果ss为空，则报告KeyError
ss.clear()	清空集合ss中所有元素

第四章 组合数据类型

4.1 列表

4.2 元组

4.3 字典

4.4 集合

4.5 实验

4.6 小结

习题

4.5.1 元组的使用

将元组(1, 2, 3, 4)中第二个元素修改为5

4.5.2 集合的使用

创建一个集合num，生成10个200以内的随机数，然后求出最大值、最小值以及总和，并将这些随机数从小到大排序，最后将结果输出。

第四章 组合数据类型

4.1 列表

4.2 元组

4.3 字典

4.4 集合

4.5 实验

4.6 小结

习题

4.6 小结

本章讲解了Python的4种组合数据类型：列表、元组、字典和集合，以及它们的特点、使用方法等。

结合前几章的知识，我们可以知道：不可变数据类型有四个：数字（Numbers）、字符串（Strings）、元组（Tuples）、集合（Sets）；元素可变的数据类型有两个：列表（Lists）、字典（Dictionary）。其中，列表是Python中使用最为频繁的数据类型之一。

一切语言的基础就是数据类型

第四章 组合数据类型

4.1 列表

4.2 元组

4.3 字典

4.4 集合

4.5 实验

4.6 小结

习题

习题：

1、简述元组和列表的相同点和不同点。

2、简述字典和集合的相同点和不同点。

感谢聆听